following invariants:

- All points in the tree have dimension `k`.
- Each level has a "*split coordinate*" index `j`, such that `0 ≤ j < k`.
- If a node `N`'s split coordinate index is `j`, then `N`'s children have split coordinate equal to `(j+1) mod k`.
- For each node `N`, with split coordinate index `j`, all nodes `L` in its left subtree have a smaller value for `N`'s split coordinate, `L[j] < N[j]`, and all nodes `R` on `N`'s right subtree have a larger or equal value for N's split coordinate, `R[j] ≥ N[j]`.

9.2.3 The Importance of Being Balanced

So far in this section we consciously ignored a detail, a special characteristic we look for when it comes to binary search trees: whether the tree is balanced or not. As for the regular `BST`, you might have already figured out that the order in which we insert elements in our k-d tree determines the shape of the tree. Even having finite areas is not a synonym of having small areas, nor good partitioning: if you look at the example in figure 9.3, we have carefully chosen points "by hand" in order to create a balanced tree, but it is easy to provide an example of a terrible insertion sequence that would create an imbalanced tree (just insert points starting from the top-left going in order towards the bottom-right corner, for instance).